

COMPLETE TECHNICAL DOCUMENTATION

SHOPORE

Shoporestore E-commerce Platform

Architecture, features, frontend, backend, SQL Server data model, API reference, setup, testing, deployment readiness, limitations, and interview guide.

Frontend

Next.js 16, React 19, TypeScript,
Tailwind CSS

Backend

ASP.NET Core Web API, .NET 10,
EF Core

Database

Microsoft SQL Server,
Shopore_DB

Table of contents

1. Executive overview

2. Scope and objectives

3. Technology stack

4. System architecture

5. Repository structure

6. Frontend design

7. Functional modules

8. Responsive design

9. State and browser storage

10. Core workflows

11. Backend design

12. API reference

13. Database design

14. Security assessment

15. Local setup

16. Testing and verification

17. Deployment plan

18. Current limitations

19. Recommended roadmap

20. Interview presentation

21. Troubleshooting

22. Commands and configuration

Purpose of this document

This is a source-grounded description of the current project. It separates completed behavior from development-only behavior and recommended production improvements.

Document status

Item	Value
Project name	Shoporestore / Shopore
Project type	Full-stack responsive e-commerce web application
Primary market assumptions	India; INR pricing; PIN-code delivery; Asia/Kolkata dates
Current operation	Local frontend + local ASP.NET API + SQL Server
Documentation date	30 June 2026

Executive overview

Shoporestore is a full-stack shopping platform built to demonstrate a complete customer journey: discovering products, filtering a catalog, viewing product details, maintaining a bag and wishlist, authenticating an account, saving addresses, paying or selecting cash on delivery, viewing orders, cancelling orders, requesting returns, reviewing purchased products, and contacting support.

The user interface is a Next.js App Router application. It communicates through a shared API bridge with a separate ASP.NET Core Web API. The backend uses Entity Framework Core with Microsoft SQL Server and contains compatibility logic for the existing `Shopore_DB` legacy schema.

Customer experience

Responsive storefront, rich categories, search, filters, cart, payments, profile and order tracking.

Business services

Catalog, account, address, order, payment, inventory, wishlist, reviews and subscriber APIs.

Data persistence

SQL Server stores core commerce records; browser storage supports fast cart and user-interface state.

Scope and objectives

Primary objectives

- Provide a modern multi-category shopping interface.
- Support desktop, tablet and mobile layouts.
- Separate frontend and backend responsibilities.
- Persist users, products, orders, payments and reviews.
- Demonstrate realistic checkout and account workflows.

Project boundaries

- No admin dashboard is exposed in the current frontend.
- Shipping carrier integration is simulated.
- Email/SMS and production OTP delivery are not connected.
- Payment verification requires security hardening before real use.
- Hosting configuration is not yet production-ready.

Overall assessment

The project is suitable as a substantial portfolio demonstration. It shows frontend composition, responsive design, API integration, relational modelling, checkout logic and legacy-database compatibility. It should be presented as a working full-stack prototype, not yet as a production commerce service.

Technology stack

Layer	Technology	Responsibility
Web framework	Next.js 16 App Router	Routing, page composition, production bundling and static/dynamic rendering.
UI runtime	React 19	Interactive components, client state and lifecycle behavior.
Language	TypeScript 5.7	Typed frontend models and safer component contracts.
Styling	Tailwind CSS 4 + component CSS	Responsive layouts, component presentation and design tokens.
UI primitives	Radix UI / shadcn-style components	Dialogs, forms, tabs, popovers, controls and accessible primitives.
Icons / feedback	Lucide React, Sonner	Icons and user notifications.
API	ASP.NET Core Web API, .NET 10	HTTP endpoints and commerce business operations.
Persistence	EF Core 10 + SQL Server	Object-relational mapping and relational storage.
API documentation	Swagger / OpenAPI	Local endpoint discovery and testing.
Payment clients	Razorpay Checkout, Google Pay JS	Checkout user experience; backend confirmation is currently prototype-level.
Analytics	Vercel Analytics	Enabled only in production builds.

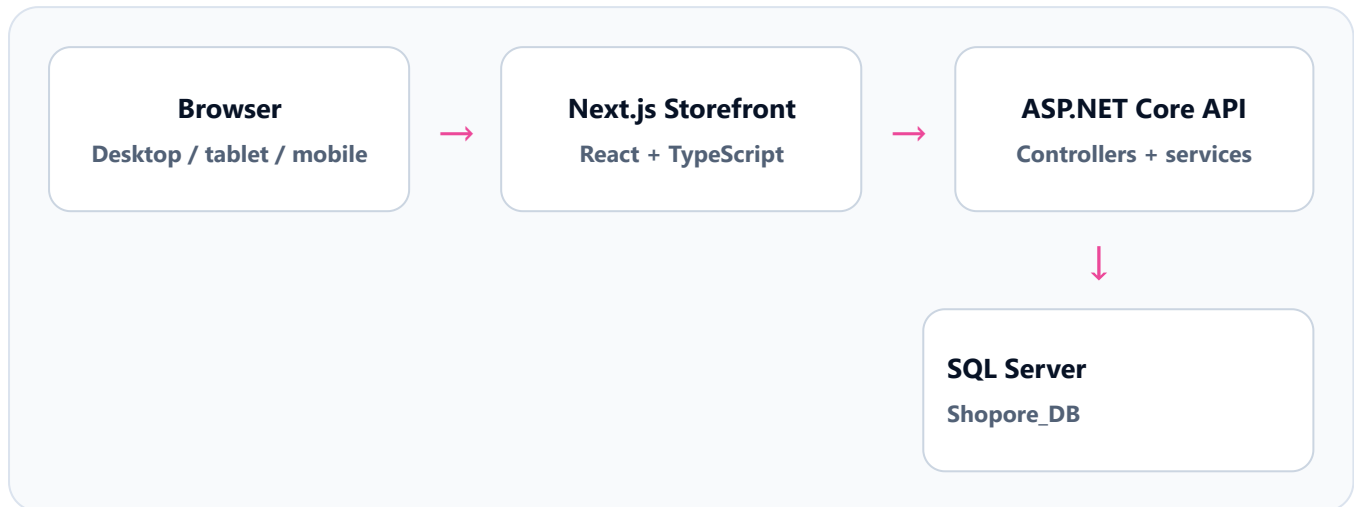
Key package characteristics

The frontend uses client components where browser storage, modal interaction and payment SDKs are required. Shared UI components are under `components/` ; reusable commerce behavior is under `lib/` . The backend uses controller-based routing, DTOs for snake_case frontend contracts, EF Core entities, response mappers and startup schema services.

Version note

The backend project targets `net10.0` . A deployment machine must have a compatible .NET runtime, or the application must be published as self-contained.

System architecture



Request flow

1. A user interacts with a page or shared component in the Next.js frontend.
2. The frontend calls `apiFetch(path, init)` from `lib/api.ts`.
3. `buildApiUrl` combines the path with `NEXT_PUBLIC_API_URL`.
4. An ASP.NET controller validates and processes the request.
5. EF Core reads or changes records in `Shopore_DB`.
6. The backend response mapper returns frontend-friendly JSON.
7. The UI updates its component state and, where applicable, its browser cache.

Separation of concerns

Frontend owns

- Rendering and navigation
- Responsive interaction
- Catalog filters and presentation
- Cart cache and optimistic wishlist state
- Payment SDK launch
- Toasts, loaders and skeletons

Backend owns

- User and profile persistence
- Address and order persistence
- Product catalog and inventory
- Payment/order records
- Verified-purchase review eligibility
- Database compatibility and seed routines

Primary integration seam

`lib/api.ts` is the durable frontend/backend boundary. Changing the API host should require updating `NEXT_PUBLIC_API_URL` , not editing every page.

Repository structure

```

Shoporestore/
├─ app/                    Next.js routes and page components
│  ├─ auth/               Login and registration
│  ├─ men, women, kids/   Fashion departments
│  ├─ home, beauty, genz/ Additional departments
│  ├─ products/[id]/      Product details and reviews
│  ├─ cart/, checkout/    Bag and checkout flow
│  ├─ profile/            Account, orders and support
│  └─ about, contact, policies Informational pages
├─ components/            Navbar, footer, listings, modals and UI primitives
├─ lib/                   API, cart, wishlist, pricing and delivery utilities
├─ data/knowledge/        FAQ and support knowledge content
├─ public/                Product, banner and branding media
├─ scripts/               Product seeding utility
├─ docs/                  Generated project documentation
└─ Shoporestore_Backend/
   └─ Shoporestore.Api/
      ├─ Controllers/      HTTP API endpoints
      ├─ Data/             EF Core DbContext
      ├─ Dtos/             Request contracts
      ├─ Models/           Catalog and commerce entities
      ├─ Services/         Mapping, schema, password and seed services
      └─ Program.cs        Dependency injection and middleware

```

Frontend route map

Area	Routes	Purpose
Landing	<code>/</code> , <code>/explore</code>	Campaigns, featured products, brands and discovery.
Departments	<code>/men/[section]</code> , <code>/women/[section]</code> , <code>/kids/[section]</code> , <code>/home/[section]</code> , <code>/beauty/[section]</code> , <code>/genz/[section]</code>	Category and subcategory product listings.
Product	<code>/products/[id]</code>	Details, sizing, delivery check, wishlist and reviews.
Commerce	<code>/cart</code> , <code>/checkout</code> , <code>/wishlist</code>	Bag, payment selection and saved products.
Account	<code>/profile</code> , <code>/profile/orders/[id]</code>	Profile, order history, status, cancellation and returns.
Authentication	<code>/auth/login</code> , <code>/auth/register</code> , <code>/verify-email</code>	Account access and development verification.

Content	/about , /careers , /contact , /help-faqs , /delivery-policy , /privacy , /terms-of-use	Company, support and policy information.
---------	---	--

Frontend design

Shared shell

`app/layout.tsx` composes the moving offer banner, responsive navigation, page content, footer, toast system, payment scripts and production analytics. The navbar contains department navigation, search, chat support, account access, notifications, bag status and a mobile drawer.

Catalog system

`ProductListingPage` provides a reusable catalog experience. Individual department route files supply configuration such as title, image directory, brands, sizes and product files. Catalog screens support:

Discovery

Departments, mega menus, landing campaigns and search.

Refinement

Brand, category, price, size, color, discount and rating filters.

Ordering

Popularity, price and other sort choices with responsive controls.

Product details

The product page retrieves backend data when available and accepts route-query fallback data for catalog compatibility. It displays pricing, discount, size selection, delivery PIN validation, product services, bag/wishlist actions and a verified-purchase review section.

Design system

The visual language uses a dark navy primary color, pink accents, light neutral surfaces, rounded cards and restrained shadows. Shared Radix-based primitives provide accessible interaction patterns while Tailwind utilities handle most responsive spacing and layout.

Asset library

The `public/` directory contains close to one thousand catalog and marketing assets across AVIF, WebP, PNG, JPG, SVG, GIF and MP4 formats. In production, these should be compressed, deduplicated and moved to object storage or a CDN.

Functional modules

Module	Implemented behavior	Status
Authentication	Email/password registration and login, phone login, OTP test flow, password reset and remembered/session login.	Prototype security
Catalog	Multiple departments, configurable sections, products, brands, prices, images, sizes, filters and sorting.	Implemented
Product details	Details, pricing, size, delivery PIN, reviews, image preview, share, wishlist and add-to-bag.	Implemented
Cart	Variant-aware keys, quantity changes, removal, totals, selected address and checkout steps.	Implemented
Wishlist	Immediate local cache plus backend synchronization for authenticated users.	Implemented
Addresses	Create, list, edit and soft-delete customer delivery addresses.	Implemented
Payments	COD, Razorpay client flow and Google Pay client flow.	Needs verification hardening
Orders	Create one order per cart item, order history, detail view, address snapshot, cancellation and order splitting.	Implemented
Returns/refunds	User-interface request tracking, cancellation/refund status presentation.	Partly browser-local
Reviews	Only eligible buyers can create/update ratings, comments and optional image data.	Implemented
Support	Help topics, contact form, FAQ knowledge and rule-based chat answers.	Delivery integrations pending
Notifications	Browser-side notification list for order, refund and account events.	Client-side
Newsletter	Email capture endpoint and footer form.	No email platform yet

Responsive design

- Mobile drawer replaces desktop department navigation below the desktop breakpoint.
- Product grids preserve two useful columns on small screens and expand at larger breakpoints.
- Filters become a touch-friendly drawer and brand filters become horizontally scrollable.
- Cart controls use a compact two-column mobile arrangement.
- Account navigation becomes a three-tab mobile control; order cards reorganize for narrow screens.
- Footer, content pages and policy typography scale without fixed-width overflow.

State and browser storage

Key / mechanism	Data	Reason
<code>accessToken</code> , <code>refreshToken</code> , <code>user</code>	Session identity	Stored in <code>localStorage</code> when “remember” is selected, otherwise <code>sessionStorage</code> .
<code>shopore-cart</code>	Cart items, selected product variant and quantity	Fast guest cart persistence and same-tab updates.
<code>shopore-wishlist</code>	Cached wishlist products	Immediate UI feedback; synchronized with the API after login.
Order snapshots	Product and address display fallback	Preserves display details when legacy order records are incomplete.
Profile avatar	Data URL	Local profile customization without server file storage.
Notification keys	Account/order events	Local notification bell behavior.
Return request key	Return tracking UI state	Prototype return workflow.

State strategy

The project intentionally combines server state with lightweight browser persistence. This makes local demos resilient when backend data is incomplete, but production deployment should move authoritative identity, notification, return and avatar data to secure backend services.

Security implication

`localStorage` and `sessionStorage` are readable by JavaScript. They are acceptable for this prototype but not ideal for long-lived production authentication. Secure `HttpOnly` cookies and server-enforced authorization are recommended.

Cross-component updates

Custom browser events such as cart, wishlist, notification and avatar update events allow the navbar and pages to refresh immediately after same-tab storage changes.

Core customer workflows

Product discovery to cart

1. User selects a department or searches.
2. Listing configuration and backend product data are normalized into cards.
3. User filters/sorts and opens `/products/[id]`.
4. User selects size and validates delivery PIN if desired.
5. `addToCart` creates a variant-aware key from product ID, image and size.
6. The bag count updates through a custom browser event.

Checkout and COD order

1. User reviews cart items and quantities.
2. User logs in and selects or creates an address.
3. User selects Cash on Delivery.
4. Frontend posts the customer, address, total and item snapshots to `POST /api/orders`.
5. The backend creates an order record per cart item and returns order DTOs.
6. Frontend clears the bag and displays order history.

Online payment order

1. Frontend requests a provider order from `POST /api/payment/create-order`.
2. Razorpay or Google Pay is opened on the client.
3. Client receives payment identifiers/token data.
4. Frontend posts payment and order data to `POST /api/payment/verify`.
5. Backend creates order and payment records together.

Current payment boundary

The backend currently checks that a payment ID exists but does not cryptographically verify the Razorpay signature. This flow is appropriate for a local demonstration only.

Verified-purchase review

1. Product review API finds an order item belonging to the user and product.
2. Cancelled/refunded orders are excluded.
3. One review is allowed per user and order item.

4. After saving, aggregate rating and review count are recalculated on the product.

Backend design

Application startup

`Program.cs` registers controllers, JSON number handling, Swagger, CORS, EF Core SQL Server and the application middleware. During startup it ensures the database exists and runs compatibility/normalization/seeding services.

Startup services

- `DatabaseSchemaService.EnsureCompatibleAsync`
- `OrderNumberService.NormalizeExistingAsync`
- `ProductSizeCatalog.SeedAsync`
- `DemoReviewSeeder.SeedAsync`

Service utilities

- PBKDF2 password hashing
- Random token generation
- Slug generation
- Response mapping
- Product-size catalog behavior

Controller groups

Auth Products Categories Stock Addresses Profile Orders Payments Wishlist Reviews
PIN code Contact Chat Subscriber

Legacy database compatibility

The backend is not designed against an empty ideal schema only. It accounts for existing columns and constraints in `Shopore_DB`. Compatibility services add or normalize required structures at startup. Response mapping and order snapshots also reduce frontend breakage when older records differ from current models.

Strong architectural decision

Order items store name, brand, image, size, color and price snapshots. Historical order display therefore does not depend entirely on the current product record.

CORS

The configured CORS policy allows local development origins: `localhost` , `127.0.0.1` and private hosts beginning with `192.168.` . Production domains must be explicitly configured before hosting.

API reference — identity and customer

Method	Route	Purpose	Important input
POST	<code>/api/auth/register</code>	Create customer and return tokens/user.	fullName, email, password, phone?
POST	<code>/api/auth/login</code>	Email/password login.	email, password
POST	<code>/api/auth/phone-login</code>	Phone/password login.	phone, password
POST	<code>/api/auth/email-otp/request</code>	Create 10-minute development OTP.	phone
POST	<code>/api/auth/email-otp/verify</code>	Verify unused OTP and return session data.	phone, otp
POST	<code>/api/auth/reset-password</code>	Reset password after matching email/phone.	phone, email, password, confirmPassword
GET	<code>/api/Auth/verify-email?token=...</code>	Development email verification response.	token
GET	<code>/api/profile?user_id=...</code>	Read profile.	user_id
PATCH	<code>/api/profile</code>	Update name, phone, birth date and gender.	user_id and profile fields
GET	<code>/api/addresses?user_id=...</code>	List non-deleted addresses.	user_id
POST	<code>/api/addresses</code>	Create delivery address.	user_id, name, phone, PIN, address, city, state
PUT	<code>/api/addresses/{id}</code>	Update owned address.	address ID and user_id
DELETE	<code>/api/addresses/{id}?user_id=...</code>	Soft-delete owned address.	address ID, user_id

Catalog, wishlist and reviews

Method	Route	Purpose
GET	<code>/api/products</code>	List active catalog products.
POST	<code>/api/products/by-images</code>	Resolve products from an image list.
GET	<code>/api/products/{id}</code>	Read product detail.
POST / PUT / DELETE	<code>/api/products</code> , <code>/api/products/{id}</code>	Create, update or deactivate catalog products.

GET / POST	<code>/api/categories</code>	List or create categories.
POST	<code>/api/stock/adjust</code>	Change variant stock and record a transaction.
GET / POST / DELETE	<code>/api/wishlist</code>	List, add or remove user wishlist products.
GET	<code>/api/reviews/product/{productId}?user_id=...</code>	Ratings summary, reviews and buyer eligibility.
POST	<code>/api/reviews</code>	Create/update a verified-purchase review.

API reference — commerce and support

Method	Route	Purpose	Important behavior
GET	<code>/api/orders?user_id=...</code>	Customer order history.	Includes item/address snapshots; excludes smoke fixtures.
POST	<code>/api/orders</code>	Create COD or direct order.	Creates one order for each submitted cart item.
PATCH	<code>/api/orders</code>	Cancel order.	Rejects cancelled, refunded or delivered orders.
POST	<code>/api/orders/{orderId}/split</code>	Split multi-item legacy order.	Uses a DB transaction and adjusts associated payment rows.
POST	<code>/api/payment/create-order</code>	Create payment-order response.	Returns generated ID, paise amount and INR currency.
POST	<code>/api/payment/verify</code>	Persist paid orders and payments.	Currently does not verify provider signature cryptographically.
GET	<code>/api/pincode/{pin}</code>	Validate six-digit PIN and delivery availability.	Currently returns a fixed Kolkata/West Bengal location response.
POST	<code>/api/contact</code>	Validate a support message.	Email delivery is not connected.
POST	<code>/api/chat</code>	Answer common commerce questions.	Rule-based keyword response.
POST	<code>/api/subscriber/subscribe</code>	Accept newsletter signup.	No external mailing-list persistence.

Representative order payload

```
{
  "user_id": 12,
  "user_email": "customer@example.com",
  "user_name": "Customer Name",
  "address_id": 4,
  "payment_method": "cod",
  "total": 1299,
  "items": [{
    "product_id": "101",
    "brand": "Shopore",
    "name": "Product name",
    "image": "/product.avif",
    "size": "M",
    "color": "Black",
    "quantity": 1,
    "price": 1299
  }]
}
```

Error conventions

Most endpoints return `{ error: "..."}` or `{ success: false, message: "..."}` for failures. The frontend API helper tolerates JSON and plain text responses and returns a connection-specific message when the backend cannot be reached.

Database design

User

1 → many Addresses, Orders, WishlistItems, Reviews

Product

1 → many Variants, Images, WishlistItems, OrderItems, Reviews

Order

many → 1 User/Address; 1 → many OrderItems; 1 → Payment

Address

Customer delivery details and soft-delete state.

ProductVariant

SKU, size, color, price and stock quantity.

OrderItem

Product reference plus historical product snapshot.

Entity	Key fields	Important constraints / behavior
User	ID, full name, email, phone, password hash, profile fields	Unique email; filtered unique phone; active flag.
Address	User ID, name, phone, PIN, address, city, state	Soft deletion and default-address flag.
Category / Brand	Name, slug, active	Unique slugs.
Product	Name, slug, category, image, price, rating	Unique slug; active flag; one-to-many variants/images.
ProductVariant	Product ID, SKU, size, color, price, stock	Unique SKU; variant-level stock.
WishlistItem	User ID, product ID	Unique user/product pair.
Order	Order number, user, address, status, totals, payment info	Unique order number; monetary precision 18,2.
OrderItem	Order/product/variant IDs, snapshot fields, quantity, prices	Protects historical display from catalog changes.
Payment	Order ID, provider IDs/signature, amount, status	Payment record linked to its order.
Review	User/product/order item, rating, comment, image	Unique user/order-item review; approval flag.
StockTransaction	Variant, quantity change, reason, reference	Inventory audit record.
LoginOtp	Phone, OTP, expiry, used	Ten-minute development OTP lifecycle.

Data modelling rationale

Variant-level price and stock are appropriate for commerce, while snapshot fields on order items ensure the customer still sees the purchased product details after catalog updates.

Security assessment

Security already present

- Passwords use PBKDF2-SHA256 with random 16-byte salt, a 32-byte key and 100,000 iterations.
- Password comparison uses constant-time cryptographic equality.
- SQL access is performed through EF Core rather than concatenated user input in normal controllers.
- SQL credentials are intended to be stored using .NET user-secrets for local work.
- CORS is limited to local/private development hosts.
- Review eligibility is checked against an owned order item.
- Address updates/deletes filter by both address and user ID.

Production security gaps

Gap	Current state	Required production change
Authentication enforcement	Random tokens are returned but not persisted or validated; no authentication middleware is configured.	Use ASP.NET Identity or signed JWT/access-refresh tokens; add <code>UseAuthentication</code> and <code>[Authorize]</code> .
Object authorization	Many endpoints trust a submitted <code>user_id</code> .	Derive user identity from authenticated claims, never from request ownership fields.
Token storage	Tokens live in local/session storage.	Prefer short-lived tokens and secure <code>HttpOnly</code> , <code>SameSite</code> cookies where appropriate.
Payment integrity	Payment signature is stored but not validated.	Verify Razorpay HMAC server-side; use provider API/webhooks; make order/payment updates idempotent.
OTP	OTP is returned in the API response.	Send through verified SMS/email provider, store a hash, rate-limit attempts and never return OTP.
Password reset	Email + phone match can reset password.	Require a single-use, expiring verified reset token.
Admin operations	Product/category/stock writes have no role protection.	Add admin roles, authorization policies and audit logging.
Rate limiting	Not configured.	Rate-limit login, OTP, contact, chat, review and payment endpoints.
Secrets	Connection-string pattern exists in settings and user-secrets guidance.	Remove committed secrets, rotate credentials and use host secret management.

Do not accept real customer payments yet

The current payment endpoints are demonstration code. Real payment processing requires server-side signature verification, webhook verification, idempotency and protected order ownership.

Local development setup

Prerequisites

- Node.js compatible with Next.js 16 and npm.
- .NET 10 SDK.
- Microsoft SQL Server with access to `Shopore_DB`.
- A browser; optional Razorpay test key for payment UI.

1. Configure SQL Server secret

```
cd D:\Shoporestore\Shoporestore\Shoporestore_Backend
dotnet user-secrets set "ConnectionStrings:ShoporeDb" \
  "Server=YOUR_SERVER;Database=Shopore_DB;User
  Id=YOUR_USER;Password=YOUR_PASSWORD;TrustServerCertificate=True;MultipleActiveResultSets=True" \
  --project .\Shoporestore.Api\Shoporestore.Api.csproj
```

2. Start the backend

```
dotnet run --project .\Shoporestore.Api\Shoporestore.Api.csproj --urls http://localhost:5285
```

Swagger is available in development at `http://localhost:5285/swagger`.

3. Configure frontend environment

```
# .env.local
NEXT_PUBLIC_API_URL=http://localhost:5285
NEXT_PUBLIC_RAZORPAY_KEY_ID=your_test_key_if_used
```

4. Install and start frontend

```
cd D:\Shoporestore\Shoporestore
npm install
npm run dev
```

Open the local URL printed by Next.js, normally `http://localhost:3000`.

Recommended startup order

1. Start SQL Server.
2. Start ASP.NET Core and confirm Swagger/products endpoint.
3. Start Next.js.
4. Register a customer and complete a COD smoke test.

Configuration rule

Do not hard-code production URLs in components. Keep all frontend API requests behind `NEXT_PUBLIC_API_URL` and `apiFetch`.

Testing and verification

Current verification commands

```
# Frontend type validation
npx tsc --noEmit

# Production compilation and route generation
npm run build

# Backend compilation
dotnet build .\Shoporestore_Backend\Shoporestore.Api\Shoporestore.Api.csproj

# Interactive API testing
http://localhost:5285/swagger
```

Recommended smoke-test checklist

Area	Test	Expected result
Responsive UI	320, 375, 768, 1024 and 1440px widths	No horizontal scroll; usable navigation and controls.
Auth	Register, duplicate registration, valid/invalid login	Correct success and error states.
Catalog	Open each department, filter, sort and product	Correct card data and stable routes.
Cart	Add two sizes, change quantity, remove item	Variant keys and totals update correctly.
Address	Create, edit, select and delete	Only owned active addresses appear.
COD	Place COD order	Order persists, bag clears, profile shows order.
Cancellation	Cancel eligible order	Status becomes cancelled and UI refreshes.
Review	Review purchased and unpurchased product	Purchased succeeds; unpurchased is rejected.
Failure	Stop API while frontend runs	Friendly connection error, no broken page.

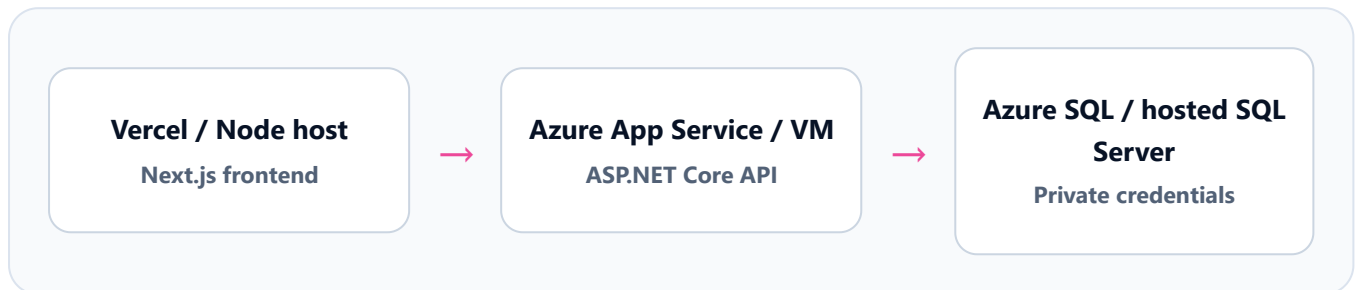
Future automated testing

- xUnit integration tests using a disposable SQL Server/Testcontainers database.
- React component tests for filters, cart calculations and authentication forms.
- Playwright end-to-end tests for registration, COD checkout and mobile navigation.
- Contract tests between frontend request types and backend DTOs.

- Security tests for IDOR, payment replay, OTP abuse and rate limits.

Deployment plan

The frontend can be hosted independently, but a live full-stack system also needs a reachable ASP.NET Core service and SQL Server database. A frontend-only deployment cannot complete login, profiles, address persistence, orders or reviews.



Deployment sequence

1. Provision hosted SQL Server and migrate/validate the schema.
2. Create a production SQL login with minimum permissions.
3. Deploy the backend with production connection string in host secrets.
4. Configure HTTPS, allowed origins, logging, health checks and backups.
5. Set `NEXT_PUBLIC_API_URL` to the public HTTPS API URL.
6. Deploy the frontend and run end-to-end smoke tests.
7. Configure real payment credentials only after verification logic is hardened.

Free/low-cost reality

Frontend hosting commonly has free tiers. Persistent ASP.NET hosting and managed SQL Server are harder to keep completely free and may sleep, expire or limit database size. For interviews, a local demonstration plus repository and this architecture document is acceptable if you clearly explain that hosting cost and production security work remain.

Interview-safe explanation

Suggested answer

"The complete project runs locally with Next.js, ASP.NET Core and SQL Server. I have not exposed the database or prototype payment/authentication flow publicly because I would first add production authentication, payment verification, secrets management and a managed SQL environment. I can demonstrate the entire flow locally and explain the deployment architecture."

Current limitations and technical debt

Area	Current limitation	Impact
Authentication	Tokens are random response strings and are not verified by the API.	Endpoints are not securely protected.
Authorization	Ownership is often supplied as user_id.	A malicious caller could attempt other IDs.
Payments	Provider order is locally generated; signature is not checked.	Not safe for real money.
OTP/email	OTP is returned; email verification/contact are placeholders.	No real delivery channel.
Shipping	PIN location and tracking timeline are simulated.	No real courier visibility.
Returns	Some return state is browser-local.	Not shared across devices and can be cleared.
Notifications	Mostly browser-local.	No push/email/SMS reliability.
Media	Large local asset collection and data URL images.	Repository/build size and performance.
Admin	No protected management UI.	Catalog/stock require API tooling.
Tests	No complete automated test suite.	Regression detection depends on manual checks.
Observability	Basic logging only.	Production diagnosis would be difficult.

Recommended roadmap

1. **Security first:** Identity/JWT or secure-cookie auth, claims-based authorization, roles, payment HMAC verification and secret rotation.
2. **Durable workflows:** backend return/refund/notification tables and real email/SMS delivery.
3. **Operations:** migrations, health checks, structured logs, backups and deployment pipeline.
4. **Quality:** integration and end-to-end tests, accessibility audit and performance budgets.
5. **Commerce depth:** admin dashboard, coupons, tax, shipping provider, inventory reservations and webhook-driven payment status.

How to present this project in an interview

60-second summary

"Shopore is a responsive full-stack e-commerce application. I built the customer storefront with Next.js, React and TypeScript, then separated the business API into ASP.NET Core with EF Core and SQL Server. It covers catalog filtering, product details, cart, wishlist, addresses, COD/online checkout, orders, cancellation and verified-buyer reviews. A key engineering challenge was making the new API work with an existing legacy SQL Server schema while keeping the frontend contract stable."

Best technical talking points

- **Architecture:** shared `apiFetch` bridge decouples frontend deployment from backend location.
- **Data integrity:** order item snapshots preserve purchase history.
- **Legacy compatibility:** startup schema services and response mapping support the existing database.
- **User experience:** consistent loaders, responsive grids, mobile drawers and touch-oriented controls.
- **Review trust:** backend checks an owned order item before allowing a review.
- **Honest engineering judgment:** identify security gaps and explain the exact production path.

Suggested demonstration order

1. Open responsive landing page and mobile navbar.
2. Browse a department; demonstrate filters and sorting.
3. Open a product, select size, verify PIN and add to bag.
4. Login/register and manage an address.
5. Place a COD order to avoid presenting prototype online payment as production-ready.
6. Open My Orders and show order detail/cancellation.
7. Show Swagger and explain SQL Server entities.
8. Finish with the security/hosting roadmap.

Questions you should be ready for

Question	Strong answer direction
Why Next.js with ASP.NET Core?	Next.js provides a rich React storefront; ASP.NET offers a typed API and strong SQL Server/EF Core integration.

Why localStorage for cart?	It provides instant guest persistence; production can merge it into a server cart after login.
How do you prevent price changes affecting orders?	Store price and product display snapshots in OrderItem at purchase time.
Is payment production-ready?	No; next step is server HMAC verification, webhooks and idempotency. The current flow is explicitly a prototype.
Why is it not live?	The three-tier system needs hosted .NET and SQL Server, plus security hardening before public exposure.

Troubleshooting guide

Symptom	Likely cause	Action
Frontend says it cannot connect	API is stopped or <code>NEXT_PUBLIC_API_URL</code> is wrong.	Start port 5285 backend; restart Next.js after environment changes.
CORS error	Frontend host is outside allowed local origins.	Add the exact production/development origin to backend CORS policy.
Backend fails at startup	Missing/invalid connection string or SQL Server unavailable.	Check user-secrets, network, SQL login and database name.
Invalid column / constraint error	Legacy schema differs from current entity assumptions.	Inspect live schema and update compatibility service before resetting data.
Old product data in order	Expected snapshot behavior.	Order history intentionally shows purchase-time details.
Wishlist returns product unavailable	Local catalog product ID does not exist in backend Products.	Seed/match the backend product or use backend-resolved product IDs.
Razorpay key missing	Environment variable absent.	Add test key to <code>.env.local</code> and restart frontend.
Order loading times out	Backend/SQL request exceeds the three-second UI budget.	Check API logs, database latency and indexes; UI keeps previous data when available.
Mobile menu overlays incorrectly	Stale dev bundle or viewport not refreshed.	Restart dev server, clear build cache if necessary, and retest at 320–640px.

Useful diagnostic endpoints

```
GET http://localhost:5285/swagger
GET http://localhost:5285/api/products
GET http://localhost:5285/api/categories
GET http://localhost:5285/api/pincode/711113
```

Do not “fix” legacy errors by deleting data first

The project deliberately uses a legacy database. Inspect the schema, constraint and payload mismatch before rebuilding tables. Compatibility changes are safer than resets when existing user/order data matters.

Commands, configuration and glossary

Frontend commands

Command	Purpose
<code>npm run dev</code>	Start development server.
<code>npm run build</code>	Create optimized production build.
<code>npm start</code>	Serve completed build.
<code>npx tsc --noEmit</code>	Type-check without emitting JavaScript.
<code>npm run seed:products</code>	Run the catalog seeding utility.

Backend commands

Command	Purpose
<code>dotnet restore</code>	Restore NuGet dependencies.
<code>dotnet build</code>	Compile backend.
<code>dotnet run --project ... --urls http://localhost:5285</code>	Run local API.
<code>dotnet user-secrets list --project ...</code>	Confirm local secrets without committing them.

Environment variables

Name	Visibility	Purpose
<code>NEXT_PUBLIC_API_URL</code>	Browser-visible	ASP.NET API base URL.
<code>NEXT_PUBLIC_RAZORPAY_KEY_ID</code>	Browser-visible public key	Launch Razorpay Checkout test UI.
<code>ConnectionStrings:ShoporeDb</code>	Backend secret	SQL Server connection string; never expose to browser.

Glossary

Term	Meaning
DTO	Data Transfer Object: request/response contract separate from database entities.

EF Core	Microsoft object-relational mapper used to access SQL Server.
Snapshot	Purchase-time copy of product/address data saved with an order.
CORS	Browser policy controlling which frontend origins may call the API.
Idempotency	Making repeated payment/order requests safe from duplicate processing.
PBKDF2	Password-based key derivation algorithm used for password hashing.

Final project statement

Shoporestore demonstrates a broad, coherent e-commerce experience and a real frontend/API/database split. Its strongest portfolio value comes from the amount of customer flow implemented, the responsive interface, and the legacy SQL Server integration. Its next milestone is production security and durable external-service integration.